

shell编程

1. 一. shell简介



Shell语言: 是一个 C 语言编写的脚本语言, 它是用户与 Linux 的桥梁

shell编程: 通常指的是学习shell命令语法后, 利用这套语法开发脚本程序, 操作、访问内核服务。而不是使用C语言去编写一个shell程序。

shell解释器: unix默认sh, Linux的默认是Bash: 全称是bash shell, bash兼容sh, 针对sh编写的Shell代码可以不加修改地在bash中运行。



标识符命名规则: 字母, 数字, 下划线以及汉字组成, 但是不建议用汉字!!!

注意事项: 标识符不能以数字开头, 不能是关键字, 尽量见名知意

2. 二. shell变量



shell中有3种变量: 用户变量、环境变量、特殊变量

用户变量在编程过程中使用量最多

环境变量主要是在程序运行时需要设置

特殊变量在对参数判断和命令返回值判断时会使用

变量的定义语法: 变量名=变量值

变量的定义需遵循的规则

- 1): 变量名可以由字母、数字和下划线组成, 但是不能以数字开头, 不能使用关键字, 严格区分大小写
- 2): 在 Bash中, 变量的默认类型都是字符串型, 如果要进行数值运算, 则必须使用特殊命令
- 3): 变量用等号"="连接值, "="左右两侧不能有空格。
- 4): 变量值建议用中如果有空格, 则需要使用单引号或双引号包含, 如 test="hello world!"
- 5): 在变量值中, 可以使用转义符"\"。

2.1. 1. 用户变量



定义普通变量: 变量名=变量值

注意: "="左右两侧不能有空格!!!

定义只读变量: readonly 变量名=变量值

注意: 只读变量生命周期窗口关闭或者脚本执行结束

获取变量值: \$变量名 或者 \${变量名}

建议: 使用\${变量名} 因为它能标识变量的边界范围

删除普通变量: unset 变量名

注意: unset不能删除只读变量

修改普通变量值: 已有变量名=新变量值

修改普通变量为只读变量: readonly 变量名

2.2. 2.环境变量



环境变量:是操作系统 (Windows、Linux、Mac) 在运行的时候, 记录的一些关键性信息, 用以辅助系统运行。

注意: 环境变量的名字习惯上使用大写字母

查看系统记录的环境变量: env

常见的环境变量: HOSTNAME SHELL USER PATH ...

自定义环境变量格式: export 变量名=变量值

临时设置: 直接命令形式编写

注意:窗口关闭就是失效

永久设置: 在/etc/profile文件中编写

注意: 编写重新登录生效或者source /etc/profile

2.3. 3.特殊变量



我们可以在执行 Shell 脚本时, 向脚本传递参数, 这时候可以使用特殊变量来获取参数

在执行Shell程序脚本时, 可以向shell脚本动态传递参数。好处是某些配置属性不用写死在脚本中。

动态传递参数的方式: sh shell脚本文件 参数1 参数2 ...

Shell常用的特殊变量在shell脚本内部支持接收参数并使用:

\$0: 获取当前脚本文件的名称

\$n: 根据数字顺序获取对应的参数 注意:n代表1,2,3,4,5...

#: 获取参数的总个数

*: 获取所有的参数内容

3. 三.字符串数据类型



在 Bash中, 变量的默认数据类型都是字符串型

定义字符串: 变量名=值 变量名='值' 变量名="值"

建议: 使用双引号定义字符串, 因为单引号括起来内容都是普通字符, 双引号括起来的内容会识别特殊含义比如: \$和`



```
[root@cent1 ~]# name=斌子
[root@cent1 ~]# age='18'
[root@cent1 ~]# gender="男人"
[root@cent1 ~]# echo $name $age $gender
斌子 28 男人
[root@cent1 ~]# echo '我的名字是${name} 年龄是${age} 性别是${gender}'
我的名字是${name} 年龄是${age} 性别是${gender}
[root@cent1 ~]# echo "我的名字是${name} 年龄是${age} 性别是${gender}"
我的名字是斌子 年龄是28 性别是男人

[root@cent1 ~]# date
Sat Mar 18 11:51:16 CST 2025
[root@cent1 ~]# echo date
date
[root@cent1 ~]# echo `date`
Sat Mar 18 11:51:29 CST 2025
[root@cent1 ~]# now1=date
[root@cent1 ~]# echo $now1
date
[root@cent1 ~]# now2=`date`
[root@cent1 ~]# echo $now2
Sat Mar 18 11:51:29 CST 2025
[root@cent1 ~]# echo '现在的日期是${now2}'
现在的日期是Sat Mar 18 11:51:29 CST 2025
[root@cent1 ~]# echo "现在的日期是${now2}"
现在的日期是Sat Mar 18 11:51:29 CST 2025
```

4. 四.shell命令和shell脚本



shell命令、shell脚本: 本质上都是属于shell编程;

shell命令: 倾向于在linux命令行中使用, 适合逻辑简单场景;

shell脚本: 倾向于在脚本文件中编写, 适合复杂逻辑处理。

注意: 1.shell脚本文件一般以.sh结尾 2.文件第一行一般为#!/bin/bash

shell脚本的执行方式:

方式一: 相对路径命令执行

格式: sh|bash|source|. 脚本

注意: 需要进入脚本的所在工作目录, 然后使用对应的sh命令来执行脚本
这种执行方式, 脚本文件不需要具有可执行权限。

方式二: 相对路径直接执行

格式: ./脚本

注意: 需要先进入到脚本所在的目录, 然后使用 ./脚本方式执行,
这种执行方式, 必须保证脚本文件具有可执行权限。

方式三: 绝对路径执行

格式: /绝对路径/脚本

注意: 需要使用脚本的绝对路径中执行, 指的是直接从根目录/到脚本目录的绝对路径, 这种执行方式, 必须保证脚本文件具有可执行权限。

shell脚本也可以动态传参: sh 脚本 参数1 参数2 ... 注意: 脚本内部可以使用特殊变量获取

4.1. 案例1:脚本中变量操作

创建一个binzi1.sh脚本文件,要求此脚本内先定义一个字符串变量str并赋值binzi666,然后打印该字符串 最后保存并执行此脚本

注意: ①shell脚本后缀名没有要求, 通常以.sh结尾

在shell中除了第一行的#表示特殊格式外, 其他地方的#符号一般表示注释。②#! 是一个约定的标记, 它告诉系统 这个脚本需要什么解释器来执行 ③相对路径方式如果不加./直接执行脚本 默认去系统环境变量中查找

```
[root@cent1 ~]# mkdir binzi
[root@cent1 ~]# cd binzi
[root@cent1 binzi]# ls
[root@cent1 binzi]# vim binzi1.sh
#!/bin/bash
str=binzi666
echo $str
# 注意保存内容
[root@cent1 binzi]# ls
binzi1.sh
[root@cent1 binzi]# sh binzi1.sh
binzi666

[root@cent1 binzi]# ./binzi1.sh
-bash: ./binzi1.sh: Permission denied
[root@cent1 binzi]# pwd
/root/binzi
[root@cent1 binzi]# /root/binzi/binzi1.sh
-bash: /root/binzi/binzi1.sh: Permission denied

[root@cent1 binzi]# chmod +x binzi1.sh
[root@cent1 binzi]# ls
binzi1.sh
[root@cent1 binzi]# ./binzi1.sh
binzi666
[root@cent1 binzi]# /root/binzi/binzi1.sh
binzi666

[root@cent1 binzi]# vi binzi1.sh
#!/bin/bash
#定义字符串
str=binzi666
#打印字符串
echo $str
[root@cent1 binzi]# ./binzi1.sh
binzi666
```

4.2. 案例2:脚本中获取参数

- 1.编写一个脚本文件binzi2.sh,在里面使用之前学习的特殊变量分别获取当前脚本文件名称,传入参数个数并分别获取对应参数内容
- 2.执行binzi2.sh脚本文件,传入3个参数分别为:张三,18,男
- 3.使用sh命令方式执行binzi2.sh脚本文件

```
[root@cent1 binzi]# vi binzi2.sh
#!/bin/bash
echo "文件名称为:$0"
echo "第一个参数为:$1"
echo "第二个参数为:$2"
echo "第三个参数为:$3"
echo "参数个数为:$#"
echo "所有参数:$*"
"binzi2.sh" [New] 7L, 174C written
[root@cent1 binzi]# sh binzi2.sh
文件名称为:binzi2.sh
第一个参数为:
第二个参数为:
第三个参数为:
参数个数为:0
所有参数:
[root@cent1 binzi]# sh binzi2.sh 斌子 18 666
文件名称为:binzi2.sh
第一个参数为:斌子
第二个参数为:18
第三个参数为:666
参数个数为:3
所有参数:斌子 18 666
```